

# Flipkart-Style E-Commerce Platform PRD

## Executive Summary

This document defines a Product Requirements Document (PRD) for a demo e-commerce marketplace modeled on Flipkart. It covers the end-to-end scope for three user roles – **Customers, Sellers, and Admins** – detailing key features, user journeys, functional and non-functional requirements, and technical architecture. The platform will include browsing/catalog, search, product details, cart, checkout, payments, order management (including fulfilment, returns, refunds), seller onboarding and management, inventory, pricing/promotions, logistics, analytics, and customer support. The PRD aligns with Flipkart-inspired best practices: customers will enjoy user-friendly browsing, personalised recommendations, and multiple payment options [52†L24-L31] ; sellers can self-onboard, list products, manage stock, view analytics and promote items [52†L24-L31] ; and admins will oversee the ecosystem via dashboards. Key non-functional targets include high availability (99.9% uptime), sub-second search performance, secure PCI-DSS compliant payments, and scalable microservices architecture [22†L49-L54] [23†L99-L105] . An industry-standard 30-week implementation timeline is estimated for core features [52†L24-L31] .

## Scope and Objectives

- **Scope:** Develop a multi-vendor online marketplace with web and mobile interfaces. The platform includes a **Customer storefront, Seller portal, and Admin console**, supporting features from user registration through purchase and fulfilment. Key modules: product catalog, search, cart/checkout, payments, orders, returns, seller onboarding, inventory, pricing/promotions, logistics integration, warehousing, analytics, customer support, and admin controls.
- **Objectives:** Enable buyers to easily discover and purchase products, enable sellers to list/manage inventory and process orders, and enable admins to manage marketplace content, users, and compliance. The platform must be reliable, performant, secure, and comply with e-commerce norms (e.g. GST invoicing in India).
- **Success Metrics:** Example KPIs include conversion rate (target ~3–5% [19†L1734-L1742] ), page load  $\leq 2$  s, 99.9% uptime [22†L49-L54] , retention rate growth, and seller retention. We will track bounce rate, average order value (AOV), customer lifetime value (CLV) [19†L1828-L1836] , and customer satisfaction (CSAT).

## Target Users & Personas

The platform has three primary user types: **Customers (Buyers), Sellers (Merchants), and Admins (Marketplace Operators)**. Personas illustrate key user segments:

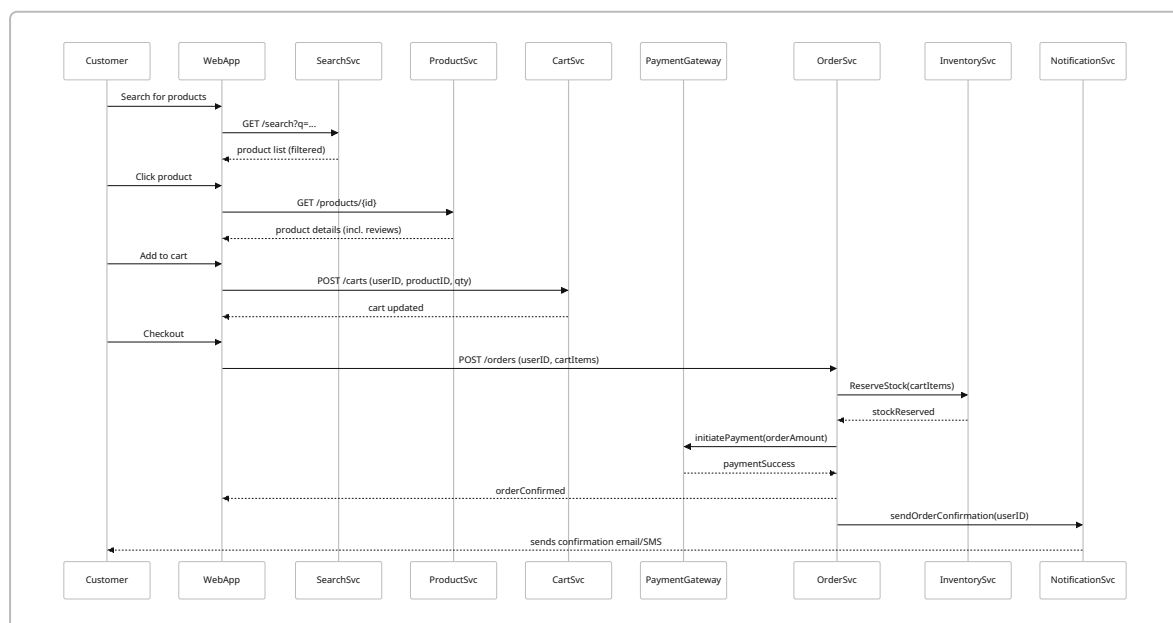
- **Customer Persona:** e.g. *"Amrita, 28, working professional"* – tech-savvy, price-conscious, shops via mobile. Values fast search, detailed reviews, and flexible payments. (Buyer personas are **fictional profiles** of ideal customers, based on research [25†L1636-L1644] [27†L59-L67] .)
- **Seller Persona:** e.g. *"Vijay, 45, small electronics retailer"* – needs to reach online customers. Expects easy onboarding, bulk upload, inventory management, and business analytics.

- **Admin Persona:** e.g. “Priya, 30, marketplace operations manager” – manages platform health, user support, policies, and promotions. Requires dashboards for metrics (revenue, orders), and controls for content and seller activities.

## User Journeys and Sequence Flows

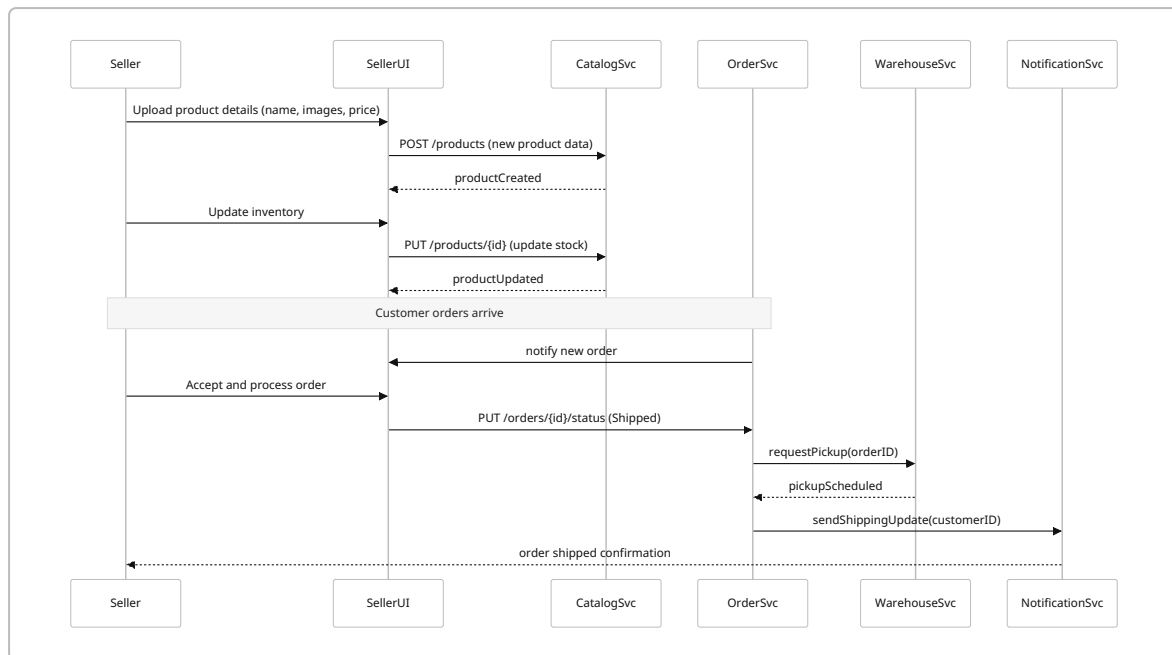
### Customer Journey (Browse → Purchase → After-sales)

Typical buyer flow: user lands on homepage (banners, recommendations), uses search or category to **find products**, views product detail (images, specs, reviews) and adds items to **shopping cart**; proceeds to checkout, enters shipping address, selects payment method, and places order. During checkout, the system reserves inventory and processes payment through a gateway (e.g. Razorpay, PayU) [40†L396-L400] . After order placement, the user sees an order confirmation, and can track status (packed, shipped, delivered). If needed, the user can initiate a return/refund via order history.



### Seller Journey (Onboarding → Catalog Management → Order Fulfilment)

Seller flow: merchant registers via Seller Portal and completes KYC. Once approved, the seller **lists products** by creating product entries (titles, descriptions, images, price, stock) [49†L267-L274] . Sellers manage inventory (update stock levels) and configure pricing/promotions. When customers place orders, the seller sees new orders in their dashboard [49†L262-L264] . The seller then processes orders: generates invoice/label, packs items, and hands over to courier. They update shipment status (or it auto-updates via partner API). On returns, the seller approves and processes refunds. Analytics (sales trends, traffic) help the seller refine catalog and promotions [49†L275-L284] .



## Admin Journey (Platform Management)

Admins manage users, products, and platform settings. Key flows: user or seller support (flag, deactivate accounts), content moderation (category/catalog management), promotions configuration (e.g. deals, coupons) and financial reconciliation (commissions). The admin also uses dashboards to monitor KPIs (sales, active users, etc). They can generate reports or audits (e.g. tax/GST reports). For example, an admin might approve a seller application: the Admin UI sends a request to the Onboarding service to verify KYC, then flips seller's status to "active".

## Feature Modules and Requirements

### Catalog & Search

- **Categories & Navigation:** Hierarchical categories (e.g. Electronics > Mobile) and landing pages.
- **Search:** Full-text search with autocomplete and filters (brand, price, rating, etc). Suggested products and synonyms. According to Flipkart's design, search filters (e.g. by color/size) and auto-suggestions with images should be supported 【49†L238-L242】 .
- **Product Detail Page:** Rich product info, images (gallery/zoom), specifications, Q&A, user reviews/ratings 【49†L251-L252】 , related and recommended products 【49†L226-L229】 . Reviews and ratings are crucial for trust 【49†L251-L252】 . The system should fetch reviews only from verified purchases and display average rating.
- **Personalization:** Show recommendations (e.g. "Customers also bought") based on browsing history and buying patterns.

### Shopping Cart & Checkout

- **Cart Features:** Persistent cart (stored in database), allow adding items from multiple sellers in one cart (Flipkart-style multi-vendor cart) 【49†L208-L211】 . The system must split the order by seller on payment. Show item thumbnails, quantities, price, estimated shipping. Enable updating quantities or removing items.

- **Checkout Flow:** Multi-step checkout: shipping address selection, apply coupon/promotion, choose shipping method (based on logistics rates), select payment method, review order. Users should be able to place cash-on-delivery (COD) orders and online payments (cards, UPI, wallets).
- **Payment Options:** Integrate multiple gateways (e.g. Razorpay, PayU) with PCI-DSS compliance [40†L396-L400] . Also support payment wallet and gift cards [49†L214-L219] . Flipkart's design includes refund to wallet if used [49†L214-L219] . The system must tokenize card data or use payment tokens for security.

## Order Lifecycle & Fulfilment

- **Order States:** Track orders through states (Placed, Confirmed, Packed, Shipped, Delivered, Cancelled, Returned, Refunded). Update states via API calls or admin actions. Notify customers of status changes (SMS/email).
- **Inventory Reservation:** On order placement, inventory is reserved/locked to prevent overselling. Use atomic transactions or distributed locks. On cancellation or failure, release stock.
- **Logistics Integration:** Connect with courier APIs (FedEx, DHL, local couriers, or Flipkart's Ekart) for shipping label generation and tracking updates. Flipkart's platform uses Google Maps API for address validation and route optimisation [40†L396-L400] . Real-time tracking info (via SMS/ push) improves transparency [39†L322-L330] .
- **Warehouse Management:** (If building own warehouses) Manage stock across warehouses. Possibly implement batch jobs to sync inventory or use a Warehouse Management System (WMS). Support features like FIFO stock, batch numbers, etc. Not mandatory for MVP.
- **Returns & Refunds:** Allow customers to initiate returns within policy time. Workflow: customer requests return, admin/seller approves, courier picks up item (with return label), upon receipt inspect item and refund. Flipkart's PRD highlights a Refund module: refunds to bank or wallet [49†L222-L225] . Define clear SLAs (e.g. refund processed within X days).

## Seller Onboarding & Catalog Management

- **Seller Registration:** Self-serve portal where merchants apply by providing business details, KYC documents (GSTIN, PAN, bank account). Admin review process with notifications.
- **Seller Dashboard:** After activation, sellers see a dashboard showing new orders, sales stats, pending actions [49†L262-L264] .
- **Product Management:** Sellers can create and manage products: enter product metadata, upload images, assign to categories [49†L267-L274] . Bulk upload via CSV may be supported. Inventory management: view and update stock levels.
- **Analytics & Reporting:** Provide basic analytics: views, conversion, sales by product and category [49†L275-L284] . Trends (most viewed, best-selling) help sellers optimize catalog. Exportable sales reports.
- **Promotions:** Enable sellers to create discounts or participate in site-wide promotions (lightbox deals, search boosts) [49†L283-L287] . Sellers should be able to apply percentage or flat discounts to products.

## Pricing & Promotions

- **Discounts/Coupons:** The platform can issue coupons (site-wide or category-specific) and support promo codes during checkout. Sellers may configure their own offers (with admin approval).
- **Dynamic Pricing:** Possibly allow price changes by sellers. Support marketplace fees/ commissions calculation.

- **Flipkart-like Deals:** Sections for “Deals of the Day”, flash sales or subscription-based savings (Flipkart First [49†L236-L237] ). Early phases may skip complex dynamic pricing beyond fixed discounts.

## Fulfilment & Logistics

- **Shipping Methods:** Offer standard and expedited shipping. Calculate shipping charges based on weight, dimensions, destination zones.
- **Logistics Partners:** Integrate with courier partners via APIs (e.g. Ekart, FedEx, Blue Dart). Automate pickup scheduling, label printing, and status updates.
- **Warehouse Routing:** If using multiple warehouses, route orders from nearest warehouse to reduce delivery time. Flipkart’s hyperlocal approach enables same-day delivery in metros [39†L310-L318] . Could plan minimal viable solution (single central warehouse) then scale.

## Analytics & Reporting

- **Business Dashboards:** Provide admins with real-time and historical dashboards: sales volume, revenue, active users, orders per second, server load, etc.
- **Operational Reports:** Reports on top-selling products, fulfillment times, inventory alerts (low stock).
- **Customer Insights:** Track repeat vs. new customers, average order value, cart abandonment rate.
- **Seller Reports:** On seller dashboard: monthly revenue, order fulfillment rate, return rates. (See Flipkart’s Seller analytics features [49†L275-L284] ).

## Customer Support

- **Helpdesk & FAQ:** Knowledge base for common issues (shipping policy, return guidelines).
- **Tickets:** In-app support ticketing: customers can submit queries or complaints. (The Medium API example had a “supports” endpoint [32†L426-L430] ).
- **Chatbot/Live Chat (Optional):** May integrate a chatbot or live agent for real-time support.

## Admin Controls & Security

- **Admin Roles & Permissions:** Role-based access control (RBAC) to restrict features. Example: only admins can approve sellers or manage all orders [46†L304-L311] . Flipkart-style RBAC ensures, for instance, that only sellers see their orders and only admins see all orders [46†L304-L311] [34†L388-L394] .
- **User Management:** Admins can view, deactivate, or blacklist users/sellers.
- **Content Management:** CMS for editing static pages (About, Terms), banners, promotions.
- **Compliance:** Ensure GST invoice generation (e-invoicing) per Indian tax laws. PCI-DSS compliance for payment data [40†L396-L400] . Data privacy (e.g. user data encryption) and adherence to regulations like GDPR or India’s IT Act.

## Security & Compliance

- **Data Security:** Encrypt sensitive data at rest and in transit (HTTPS/TLS). Secure user passwords with hashing (bcrypt). Protect against OWASP Top 10 (SQLi, XSS, CSRF).
- **Authentication:** Use OAuth/JWT tokens for session management. Support multi-factor authentication (optional) for admin accounts.
- **Payment Security:** Comply with PCI-DSS: do not store full card details, use tokenization or direct gateway redirects [40†L396-L400] .

- **Logging & Monitoring:** Log all critical actions (logins, transactions). Monitor for suspicious activity (e.g. rate-limit logins). Use tools like Prometheus/Grafana for real-time alerts.

## Scalability, Performance & Availability

- **Microservices Architecture:** To scale modules (cart, search, orders) independently, adopt microservices (or modular services) with containers (Docker, Kubernetes) 【39†L340-L348】 【13†L293-L302】 . This supports independent deployments and horizontal scaling 【13†L293-L302】 【13†L328-L337】 .
- **Caching/CDN:** Use in-memory caches (Redis/Memcached) for hot data (e.g. sessions, frequent queries) to reduce DB load 【39†L340-L348】 . Employ a CDN (e.g. Cloudflare) for static assets and fast image delivery.
- **High Availability:** Deploy across multiple availability zones or regions. Aim for  $\geq 99.9\%$  uptime 【22†L49-L54】 . Use load balancers, health checks, and auto-scaling groups. Implement failover database clusters.
- **Performance Targets:** E.g. product search results within 2–3 seconds for 90% of queries 【23†L152-L153】 . Page response times  $< 2$  s under normal load; able to handle peak loads (e.g. flash sales) with horizontal scaling.
- **Monitoring and Alerting:** Use centralized monitoring (New Relic, Datadog) to track latency, errors, throughput 【39†L363-L367】 . Set up alerts for CPU/memory spikes, error rates. Implement chaos engineering practices as Flipkart does to ensure resilience 【39†L361-L370】 .

## Backup & Disaster Recovery

- **Data Backups:** Regular backups of databases and file storage (daily/differential backups). Keep backups in separate region. Test restore procedures periodically.
- **Disaster Recovery (DR):** Maintain standby clusters or cold backups in a separate region. In case of major outage, failover to DR site. Ensure RPO/RTO are documented (e.g. RPO=1h, RTO=2h). Redundant services and automated failover scripts minimize downtime 【22†L49-L54】 【22†L98-L101】 .

## Data Model (ER Diagram)

Below is a simplified Entity-Relationship (ER) diagram. Key entities include **CUSTOMER**, **SELLER**, **PRODUCT**, **ORDER**, **ORDER\_ITEM**, **CART** and **CART\_ITEM**, with relationships to capture purchases and listings.

```
erDiagram
    CUSTOMER {
        int CustomerID PK "Primary key"
        string Name
        string Email
        string Address
    }
    SELLER {
        int SellerID PK
        string Name
        string ContactInfo
    }
    PRODUCT {
        int ProductID PK
```

```

    string Name
    decimal Price
    int Stock
}
ORDER {
    int OrderID PK
    datetime OrderDate
    string Status
}
ORDER_ITEM {
    int Quantity
    decimal Price
}
CART {
    int CartID PK
    datetime CreatedAt
}
CART_ITEM {
    int Quantity
}
// Relationships
CUSTOMER ||--o{ ORDER : places
ORDER ||--|{ ORDER_ITEM : contains
PRODUCT ||--o{ ORDER_ITEM : included_in
CUSTOMER ||--o{ CART : owns
CART ||--|{ CART_ITEM : includes
PRODUCT ||--o{ CART_ITEM : added_as
SELLER ||--o{ PRODUCT : lists

```

## APIs and Integration Points

We will expose RESTful APIs (JSON over HTTPS) for frontend consumption. Examples include:

- **Catalog APIs:** GET /products (with query params for search/filters), GET /products/{id} [34†L400-L408] .
- **Cart APIs:** POST /carts (add/update item), PUT /carts (change qty), DELETE /carts/{itemId}, GET /carts (view cart) [36†L344-L348] .
- **Order APIs:** POST /orders (place order), GET /orders (list orders), GET /orders/{id} [34†L385-L394] , PUT /orders/{id}/status (admin update), POST /orders/{id}/refund [34†L395-L398] .
- **Product Management:** POST /products (seller adds), GET /products , GET /products/{id} , PUT /products/{id} , DELETE /products/{id} [34†L405-L414] .
- **Authentication:** POST /users/login , POST /sellers/login , issuing JWT tokens.
- **Third-Party Integrations:** Payments (e.g. Razorpay/PayU SDK webhooks), SMS/Email gateways, courier APIs (for shipping and tracking), tax/GST calculation service. All interactions are via secure REST calls or SDKs.
- **OpenAPI (Swagger) Documentation:** The APIs will be documented using OpenAPI (Swagger) so developers can explore endpoints [32†L440-L444] .

## Module-Level Requirements (Summary Table)

| Module                              | Key Functional Req'ts                                                                                                                                                 |
|-------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Product Catalog &amp; Search</b> | User can browse categories and filters; full-text search with auto-suggest; product detail page shows images, specs, and reviews 【49†L226-L229】 【49†L238-L242】 .      |
| <b>Shopping Cart</b>                | Cart holds multiple items (from multiple sellers) 【49†L208-L211】 ; users can update qty or remove items; cart persists per user/session.                              |
| <b>Checkout &amp; Payments</b>      | Supports multi-step checkout; integration with multiple payment gateways (card, UPI, wallets, COD) 【40†L396-L400】 ; SSL encryption; PCI-DSS compliance.               |
| <b>Order Management</b>             | Order creation, status updates (placed, shipped, delivered, etc.); sends email/SMS updates; supports returns/refunds flow (refund to bank or wallet) 【34†L395-L398】 . |
| <b>Seller Onboarding</b>            | Self-registration with KYC; admin approval; creates seller profile and store page.                                                                                    |
| <b>Inventory Management</b>         | Sellers can set and update stock levels; system auto-reduces stock on orders; alerts on low stock.                                                                    |
| <b>Pricing &amp; Promotions</b>     | Admin and sellers can create discounts/coupons; dynamic pricing (Flash sales) optional.                                                                               |
| <b>Fulfilment &amp; Logistics</b>   | Integrates with courier APIs; label printing; real-time tracking (via SMS/push) 【39†L322-L330】 ; handles shipping status updates.                                     |
| <b>Analytics &amp; Reporting</b>    | Business dashboard for revenue, orders, traffic; seller dashboard for sales analytics 【49†L275-L284】 ; exportable CSV reports.                                        |
| <b>Customer Support</b>             | In-app support ticketing and FAQ.                                                                                                                                     |
| <b>Admin Controls</b>               | RBAC for system modules 【46†L304-L311】 ; manage users/sellers/products; monitor KPIs; content/coupon management; role permissions.                                    |
| <b>Security &amp; Compliance</b>    | HTTPS; data encryption; OWASP protections; user session timeout; GDPR/GST and PCI compliance.                                                                         |
| <b>Scalability/NFR</b>              | 99.9% uptime; handle >1000 req/sec; search <3s for 90% queries 【23†L152-L153】 ; CDN, auto-scaling; monitoring/alerts 【39†L363-L367】 .                                 |
| <b>CI/CD &amp; Testing</b>          | Automated build/test/deploy pipeline; unit/integration/E2E tests; performance testing under load; staging environment.                                                |

(Table: Modules vs. key requirements. See feature list references 【52†L24-L31】 【49†L208-L211】 .)



# Technical Architecture & Tech Stack

## Architecture Overview

We recommend a microservices-based architecture: separate services for **User/Accounts, Catalog, Search, Cart, Order, Inventory, Payments, Notifications**, etc. Each service has its own database (polyglot persistence) and communicates via REST or messaging (Kafka/RabbitMQ) for events (e.g. order created). This mirrors Flipkart's design for scalability [39†L340-L348] . A gateway/load-balancer routes requests to services. Kubernetes or similar platforms manage containerised deployments. As Flipkart does, use horizontal scaling, caching (Redis/Memcached), and async processing (queues) for resilience [39†L340-L348] .

## Deployment Architecture

- **Cloud Infrastructure:** Use AWS/Azure/GCP with multiple Availability Zones. Services run in containers (Docker) orchestrated by Kubernetes (or managed service) for auto-scaling.
- **Databases:** Relational (e.g. MySQL/Postgres) for transactional data (orders/users) and NoSQL (MongoDB/Cassandra) or search engine (Elasticsearch) for product catalog and search indices. Flipkart uses MySQL/Postgres for transactions and Elasticsearch for search [39†L392-L401] .
- **Storage:** Object storage (e.g. S3) for product images and static assets. A CDN in front of images/JS/CSS to speed up content delivery.
- **Monitoring & Logging:** Centralized logging (ELK stack or Splunk) and metrics collection (Prometheus/Grafana) as described earlier.

## Tech Stack Options (Trade-offs)

| Layer / Component | Option A             | Option B                                | Pros/Cons                                                                                                                                                                              |
|-------------------|----------------------|-----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Frontend          | React.js + Next.js   | Angular or Vue.js                       | React/Next is widely used and SEO-friendly. Angular offers structure (but heavier). Vue is easier for small teams. React has rich ecosystem (Redux, hooks).                            |
| Backend           | Java (Spring Boot)   | Node.js (Express/Nest)                  | Java Spring is mature and robust (threaded, strong typing) [39†L388-L392] . Node.js is event-driven (async), faster to develop prototypes, easy JSON handling. Can combine (polyglot). |
| Database          | SQL (Postgres/MySQL) | NoSQL (MongoDB/Cassandra)               | SQL ensures ACID for orders; NoSQL scales easily for catalog/search. Could use SQL for core & NoSQL/ES for product/search. Flipkart uses both SQL and NoSQL [39†L392-L401] .           |
| Search Engine     | Elasticsearch        | Apache Solr / Database Full-text Search | Elasticsearch is fast, scalable, built for e-commerce. Solr similar but heavier to configure. Using DB-only search is slower and lacks analytics.                                      |

| Layer / Component              | Option A                         | Option B                    | Pros/Cons                                                                                                                                                                |
|--------------------------------|----------------------------------|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Caching</b>                 | Redis                            | Memcached                   | Both are in-memory; Redis has richer data structures (lists, pub/sub) and persistence options; Memcached is simpler. Flipkart uses both Redis/Memcached [39†L341-L348] . |
| <b>Payment Gateway</b>         | Razorpay / PayU (India-specific) | Stripe / Braintree (global) | Razorpay/PayU handle Indian payment methods (UPI, netbanking, wallets). Stripe is developer-friendly (but limited UPI). Multi-gateway support is ideal.                  |
| <b>Messaging</b>               | Kafka / RabbitMQ                 | HTTP webhooks / Polling     | Kafka/RabbitMQ for decoupled async tasks (order events, emails). HTTP integration simpler but less scalable. Flipkart uses Kafka for order queuing [39†L340-L348] .      |
| <b>CI/CD</b>                   | GitHub Actions / GitLab CI       | Jenkins / CircleCI          | Managed CI (GitHub/GitLab) is easier to set up. Jenkins offers more customization. Key is automated pipelines.                                                           |
| <b>Container Orchestration</b> | Kubernetes (EKS/GKE/AKS)         | Docker Compose / On-prem    | K8s for scalability and multi-AZ. Docker Compose simpler but single-host (not for production). Cloud-managed K8s offers resilience.                                      |

(Table: Example tech stacks with pros/cons. Choices should consider team expertise, scale, and ecosystem.)

## Testing Strategy

- **Unit Tests:** Cover business logic of services (e.g. order calculation).
- **Integration Tests:** Test APIs end-to-end (e.g. placing an order through services). Use tools like Postman or automated scripts. The example API project tested endpoints via Postman [32†L508-L517] .
- **End-to-End (E2E) Tests:** Use frameworks (e.g. Selenium, Cypress) to simulate user flows (search → add to cart → checkout).
- **Performance Testing:** Load test the search and checkout processes (simulate peak traffic). Ensure the system handles 5–7× normal load (as Flipkart did for flash sales [39†L369-L377] ).
- **Security Testing:** Penetration testing for vulnerabilities; automated scans for OWASP threats.
- **User Acceptance Testing:** Beta release to sample users and sellers, gather feedback on usability and functionality.

## Non-Functional Requirements (NFRs)

Key NFR categories include **performance**, **scalability**, **availability**, **reliability**, **security**, and **usability** 【23†L99-L105】 .

- **Performance:** Page load time  $\leq 2$  seconds (desktop) and  $\leq 3$  s (mobile) for 90% of requests. Product search latency  $\leq 3$  s for 90% of queries 【23†L152-L153】 . Database operations should complete within defined SLAs (e.g. 100 ms).
- **Scalability:** The system should handle at least 10× normal traffic burst (horizontal scaling), with auto-scaling policies. It should support tens of thousands of concurrent users.
- **Availability:** Target 99.9% uptime 【22†L49-L54】 . Multi-region deployment and failover will limit downtime to a few hours/year.
- **Reliability:** Mean Time Between Failures (MTBF) and Mean Time To Recovery (MTTR) should meet SLA (e.g. MTTR < 1 hour). Backups and DR plans ensure data loss is minimal.
- **Security:** All traffic encrypted (HTTPS), comply with PCI-DSS for payments 【40†L396-L400】 , regular security audits. Role-based access prevents privilege escalation 【46†L304-L311】 .
- **Maintainability:** Code should be modular and well-documented. Configuration management (infrastructure as code).
- **Usability:** Intuitive UI/UX; accessibility standards (contrast, keyboard nav). The onboarding flow should guide new customers/sellers (typical app onboarding best practices 【6†L345-L354】 ).
- **Compatibility:** Support modern browsers, responsive design for mobile.

## KPIs and Metrics

The following key performance indicators (KPIs) will guide success:

- **Conversion Rate:** % of visitors who place orders (industry ~3% 【19†L1734-L1742】 ).
- **Average Order Value (AOV):** Dollars per order.
- **Customer Retention Rate:** % customers making repeat purchases 【19†L1830-L1839】 .
- **Customer Acquisition Cost (CAC):** Marketing spend per new customer.
- **Cart Abandonment Rate:** % users who leave without purchase.
- **Order Fulfilment Time:** Time from order placement to dispatch.
- **Uptime & Latency:** System availability (target  $\geq 99.9\%$ ) 【22†L49-L54】 , API response times.
- **Error Rates:** 5xx and 4xx error percentages.
- **Load Handling:** Peak requests/sec supported (should exceed expected peak by margin).

## Acceptance Criteria

Each feature must have clear “Definition of Done” conditions 【50†L1828-L1836】 . Examples:

- **Search:** Under normal load (100 users/min), 90% of search queries return results within 3 s 【23†L152-L153】 .
- **Checkout:** A customer can complete purchase with valid payment (successful order creation, payment capture, inventory reduction). The shopping cart maintains items between sessions.
- **Seller Onboarding:** On successful KYC submission and admin approval, a new seller account is active and can list products.
- **Security:** Verified by tests, user passwords are never returned by APIs; admins can only access management endpoints.
- **Scalability:** Under simulated 5× traffic (through load tests), response times meet SLA, with no service crashes.

- **Backup/DR:** In a simulated outage, data can be restored and services failover in <2 hours.

*(Acceptance criteria are predefined requirements ensuring each feature meets user needs 【50†L1828-L1836】 .)*

## Roadmap & MVP Scope

We propose a phased rollout:

- **Phase 1 (MVP, ~12 weeks):** User registration/login; product catalog and basic search; product detail pages; shopping cart and checkout; single payment gateway (e.g. one credit card option); order placement; admin dashboard for products/orders. No seller self-service yet (products seeded by admin). Essential NFRs in place.
- **Phase 2 (Weeks 13–24):** Add seller portal (self-onboarding, product listing, inventory updates, order notifications) 【52†L24-L31】 【49†L267-L274】 ; multi-seller cart; multiple payment options (wallet, UPI, COD); basic promotions (coupons); return/refund workflow; user and order analytics dashboards.
- **Phase 3 (Weeks 25–30):** Advanced features – full logistics integration (auto label, tracking); recommendation engine; mobile apps; loyalty program; comprehensive admin controls (role perms, content CMS); enhance performance (auto-scaling, caching).
- **MVP Scope:** Core shopping experience for customers (browse → buy) and basic seller operations.

This phased approach mirrors Flipkart’s feature timeline (~30 weeks total) 【52†L24-L31】 .

**Downloadable PDF:** A complete PRD document (with figures and appendices) is prepared as a PDF for distribution.

**Sources:** This PRD references industry best practices, Flipkart-inspired feature lists 【52†L24-L31】 【49†L208-L211】 , and technical architecture insights 【13†L293-L302】 【40†L396-L400】 . Each section has citations to authoritative sources.

---